Level 3 · about 6 h · Python 3.8+

Project: Unit converter CLI with tested pure functions
Examples run on CPython 3.12.3. **You will be able to:**

- Define functions with positional, keyword, default and variadic parameters
- Explain the LEGB rule and predict which name a line of code sees
- Avoid the mutable-default-argument trap
- Write a docstring that follows PEP 257 and a signature with PEP 484 hints

Functions and Scope

Why functions

A function gives a block of code a name so you can reuse it and test it alone. Keep each function doing one thing and returning a value instead of printing it.

Defining and calling

```
def area(width, height=1):
    """Return the area of a rectangle."""
    return width * height

print(area(3, 4))
print(area(5))
print(area(height=2, width=6))
```

```
Output (CPython 3.12.3):
12
5
12
```

- `width` is a required parameter; `height` has a default value.
- Keyword arguments let you pass values by name, in any order.

Variadic parameters

*args collects extra positional arguments into a tuple; **kwargs collects extra keyword arguments into a dict.

```
def describe(*args, **kwargs):
    return len(args), sorted(kwargs)

print(describe(1, 2, 3, unit="cm", colour="red"))
```

```
Output (CPython 3.12.3):
(3, ['colour', 'unit'])
```

The LEGB rule

Python looks a name up in this order: Local, Enclosing function, Global, Built-in. Assigning to a name inside a function makes it local to that function unless you declare global or nonlocal.

```
x = "global"

def outer():
    x = "enclosing"
    def inner():
        nonlocal x
        x = "changed by inner"
    inner()
    return x

print(outer())
print(x)
```

```
Output (CPython 3.12.3):
changed by inner
global
```

The mutable default trap

Default values are evaluated once, when the def runs, not on each call.

```
def add_item(item, bucket=[]):
    bucket.append(item)
    return bucket

print(add_item("a"))
print(add_item("b"))

def add_item_safe(item, bucket=None):
    if bucket is None:
        bucket = []
    bucket.append(item)
    return bucket

print(add_item_safe("a"))
print(add_item_safe("b"))
```

```
Output (CPython 3.12.3):
['a']
['a', 'b']
['a']
['b']
```

Docstrings and hints

PEP 257 describes docstring conventions: a short summary line, then detail if needed. PEP 484 defines the type-hint syntax. Hints are not enforced at run time; a checker such as mypy or pyright reads them.

```
def celsius_to_fahrenheit(c: float) -> float:
    """Convert degrees Celsius to degrees Fahrenheit."""
    return c * 9 / 5 + 32

print(celsius_to_fahrenheit(100))
```

```
Output (CPython 3.12.3):
212.0
```

Check yourself

1. What does `area(height=2, width=6)` return?
2. Why does `add_item("b")` print `['a', 'b']`?

3. Which scope does a name assigned inside `inner` belong to without `nonlocal`?

Project: Unit converter CLI

Build three pure functions (km to miles, kg to pounds, Celsius to Fahrenheit), add docstrings and hints, then write four `pytest` tests. Stretch: accept values with `argparse`.

Sources

- Python tutorial: More on defining functions
- PEP 257 Docstring Conventions
- PEP 484 Type Hints
- PEP 8 Style Guide for Python Code
- Python reference: Naming and binding

Code © CypherMonkeyLabs, MIT. Text CC BY-NC-SA 4.0. Educational material, not accredited coursework; no outcome or income promised. Built 2026-10-18.